



**University of Asia Pacific**  
**Department of Computer Science and**  
**Engineering**

**Compiler Design**  
**( CSE 430 )**

**Submitted By**

Mazharul Islam Sourav

ID: 22101133

Section: C-2

Semester: 4-2

**Submitted To**

Md Shafayatul Haque

Lecturer

Department of Computer Science  
and Engineering

University of Asia Pacific

# Work 05: Three Address Code Generation.

## Code –

Tokenization Function –

```
1  operators = {'+': 1, '-': 1, '*': 2, '/': 2, '%': 2, '^': 3, 'u-': 4}
2  temp_count = 1
3
4  def precedence(op):
5      return operators.get(op, 0)
6
7
8  def tokenize(expression):
9      tokens = []
10     i = 0
11
12     while i < len(expression):
13         char = expression[i]
14
15         if char == ' ':
16             i += 1
17             continue
18
19         if char.isalnum():
20             operand = ""
21             while i < len(expression) and expression[i].isalnum():
22                 operand += expression[i]
23                 i += 1
24             tokens.append(operand)
25             continue
26
27         tokens.append(char)
28         i += 1
29
30     return tokens
31
```

## PostFix Function –

```
32
33 def postfix(tokens):
34     stack = []
35     postfix = []
36
37     for i, token in enumerate(tokens):
38
39         if token not in '+-*/%^()':
40             postfix.append(token)
41
42         elif token == '(':
43             stack.append(token)
44
45         elif token == ')':
46             while stack and stack[-1] != '(':
47                 postfix.append(stack.pop())
48             stack.pop()
49
50         elif token == '-' and (i == 0 or tokens[i-1] in '+-*/%^'):
51             stack.append('u-')
52
53         else:
54             while stack and stack[-1] != '(':
55                 if token == '^' or stack[-1] == 'u-':
56                     if precedence(stack[-1]) > precedence(token):
57                         postfix.append(stack.pop())
58                     else:
59                         break
60                 else:
61                     if precedence(stack[-1]) >= precedence(token):
62                         postfix.append(stack.pop())
63                     else:
64                         break
65
66             stack.append(token)
67
68     while stack:
69         postfix.append(stack.pop())
70
71     return postfix
72
```

## Intermediate Code Generation Function –

```
73
74 def TAC(postfix):
75     global temp_count
76
77     stack = []
78     tac = []
79
80     for token in postfix:
81
82         if token == 'u-':
83             operand = stack.pop()
84             temp = f"T{temp_count}"
85             temp_count += 1
86             tac.append(f"{temp} = uminus {operand}")
87             stack.append(temp)
88
89         elif token in operators:
90             right = stack.pop()
91             left = stack.pop()
92             temp = f"T{temp_count}"
93             temp_count += 1
94             tac.append(f"{temp} = {left} {token} {right}")
95             stack.append(temp)
96
97         else:
98             stack.append(token)
99
100     return tac
101
```

## Main Function –

```
102
103 filename = input("Enter filename (e.g., input.txt): ")
104
105 try:
106     with open(filename, 'r') as file:
107         expression = file.read().strip()
108
109     print("Input Expression:")
110     print(expression)
111     print()
112
113     tokens = tokenize(expression)
114     postfix = postfix(tokens)
115     tac_code = TAC(postfix)
116
117     print("Three Address Code:")
118     for i, line in enumerate(tac_code, 1):
119         print(f"({i}) {line}")
120
121 except FileNotFoundError:
122     print(f"Error: File '{filename}' not found!")
123 except Exception as e:
124     print(f"Error: {e}")
```

## Input –

1     -(a\*b)+(c+d)-(a+b+c+d)

# Output -

```
hpaBepoowan D:\4-2\Compiler Lab\Files | 10:32:55
$ python -u "d:\4-2\Compiler Lab\Files\22101133_Labwork_06.py"
Enter filename (e.g., input.txt): Test.txt
Input Expression:
-(a*b)+(c+d)-(a+b+c+d)

Three Address Code:
(1) T1 = a * b
(2) T2 = uminus T1
(3) T3 = c + d
(4) T4 = T2 + T3
(5) T5 = a + b
(6) T6 = T5 + c
(7) T7 = T6 + d
(8) T8 = T4 - T7

hpaBepoowan D:\4-2\Compiler Lab\Files | 10:33:06
$
```

Code Link : [GitHub](#)